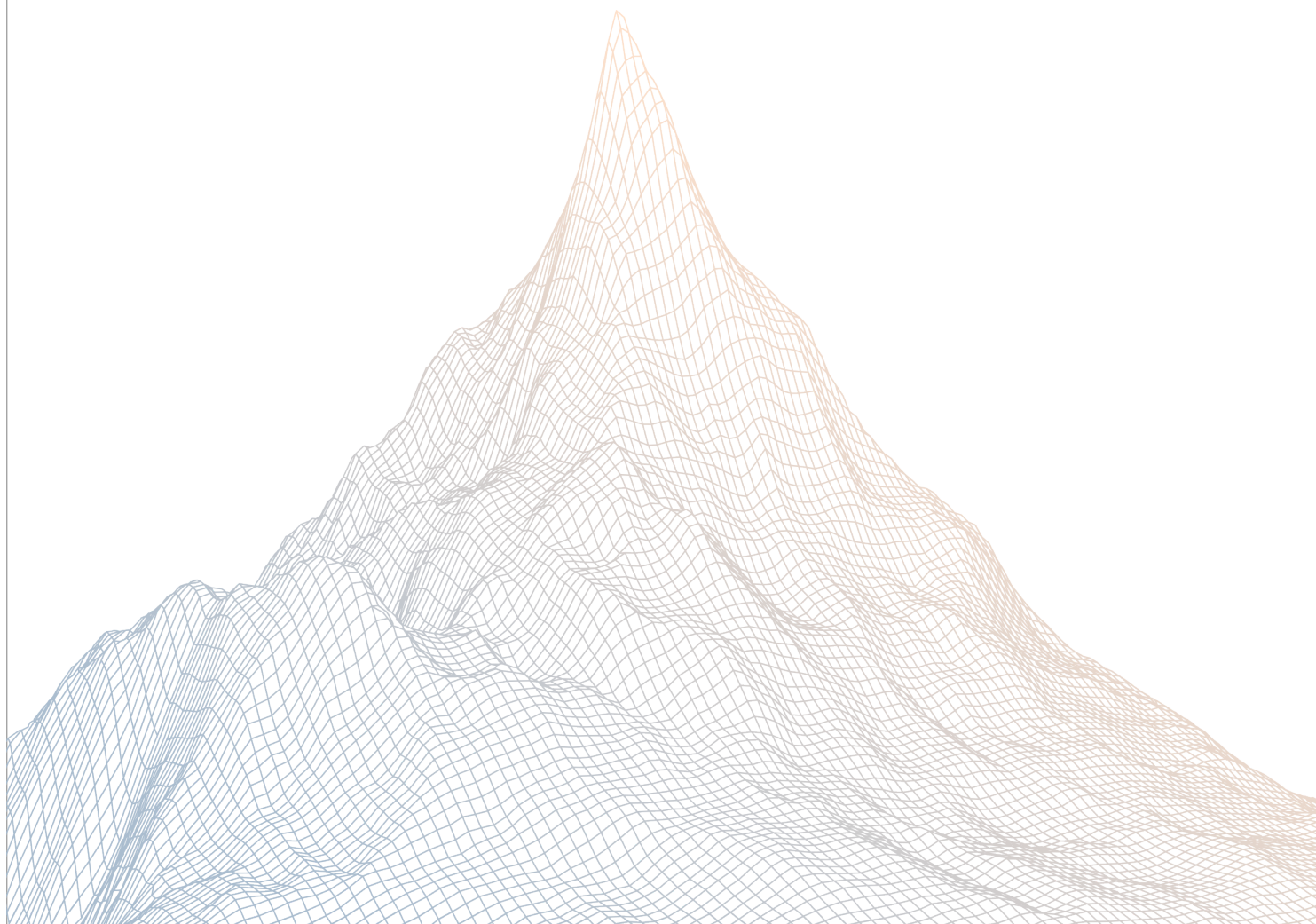


Meteora

Smart Contract Security Assessment

VERSION 1.1



Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Meteora	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Medium Risk	7
4.2	Low Risk	9
4.3	Informational	20

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	Meteora Zap Program
---------------	---------------------

Repository	https://github.com/MeteoraAg/zap-program
-------------------	---

Commit Hash	0ec22498bb28759be13b956a16f6b5652d25eded
--------------------	--

Files	src/*
--------------	-------

Target	Meteora Zap Program Mitigation Review
---------------	---------------------------------------

Repository	https://github.com/MeteoraAg/zap-program
-------------------	---

Commit Hash	db5e207a0a741c0270a6b6347a3c87355cc42eb7
--------------------	--

Files	src/*
--------------	-------

2.3 Audit Timeline

November 19, 2025	Audit start
November 25, 2025	Audit end
December 2, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	7
Informational	5
Total Issues	13

3

Findings Summary

ID	Description	Status
M-1	Incorrect rounding on <code>get_price_change_bps</code> will lead to slippage not working	Resolved
L-1	<code>min_swap_amount</code> and <code>max_swap_amount</code> are off by one in <code>calculate_swap_amount()</code>	Resolved
L-2	Zap won't allow for referral	Acknowledged
L-3	<code>calculate_swap_result()</code> should throw an error on zero output amount	Resolved
L-4	<code>calculate_swap_amount()</code> uses the <code>swap_amount_in</code> that fails <code>validate_swap_result</code>	Resolved
L-5	Potential overflow for <code>amount_in</code> in <code>calculate_swap_amount()</code>	Resolved
L-6	Usage of zap/ledger with multiple pools simultaneously is impossible	Acknowledged
L-7	Overflow possible in <code>update_ledger_balances</code>	Resolved
I-1	Typo in <code>utils</code> folder	Resolved
I-2	Typo in <code>damm_v2_utils</code>	Resolved
I-3	Bps Max is defined twice	Resolved
I-4	<code>calculate_swap_result()</code> hardcodes <code>has_partner = false</code> but does not verify it	Acknowledged
I-5	<code>pow()</code> fails to handle the case <code>exp == i32::MIN</code>	Resolved

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] Incorrect rounding on `get_price_change_bps` will lead to slippage not working

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High

Target

- /programs/zap/src/ultils/damm_v2_ultils.rs#L492

Description:

When calling the `zap_in_damm_v2` instruction, the user can provide the `max_sqrt_price_change_bps` value. This value limits how much the price can move on the swap during the zap process. This is then checked at the end of the instruction.

```
let sqrt_price_change_bps = get_price_change_bps(pre_sqrt_price,
    post_sqrt_price)?;
require!(
    sqrt_price_change_bps <= max_sqrt_price_change_bps,
    ZapError::ExceededSlippage
);
```

The actual bps change will be provided by the `get_price_change_bps()` function.

```
pub fn get_price_change_bps(pre_sqrt_price: u128, post_sqrt_price: u128) ->
    Result<u32> {
    let price_diff = if pre_sqrt_price > post_sqrt_price {
        pre_sqrt_price.safe_sub(post_sqrt_price)?
    } else {
        post_sqrt_price.safe_sub(pre_sqrt_price)?
    };

    let price_diff_prod
    = U192::from(price_diff).safe_mul(U192::from(MAX_BASIS_POINT))?;
```

```
let price_diff_bps
= price_diff_prod.safe_div(U192::from(pre_sqrt_price))?;
Ok(price_diff_bps
  .try_into()
  .map_err(|_| ZapError::TypeCastFailed)?)
}
```

The calculation, in the end, is as follows.

$$priceDiffBps = \text{floor}(\frac{priceDiff}{preSqrtPrice})$$

As this is rounded down, it will return a lower-than-real bps value. For an exemplary value of 5.9 BPS, the function will return 5BPS. If the user has now set his maximum to 5BPS, the call would pass, even though the actual change in price is 5.9BPS, which is significantly larger than the user's max of 5BPS.

Recommendations:

We recommend ceiling the calculation.

Meteora: Resolved with [PR-37](#).

Zenith: Verified.

4.2 Low Risk

A total of 7 low risk findings were identified.

[L-1] `min_swap_amount` and `max_swap_amount` are off by one in `calculate_swap_amount()`

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [damm_v2_utils.rs#L414-L438](#)

Description:

The binary search within `calculate_swap_amount()` will not converge if the search interval is 1.

That's because the update to `min_swap_amount` and `max_swap_amount` are off by one.

- Suppose `min_swap_amount = 1` and `max_swap_amount = 2`.
- That means `swap_amount = (1 + 2) / 2 = 1`.
- If we have `ExceededA` for `TradeDirection::AtoB`, it will set `min_swap_amount = swap_amount = 1`, which is the same as the previous value. So the binary search will keep continue to meaninglessly loop and search with the same `swap_amount` until it hits 20 iterations.

It should instead update `min_swap_amount = swap_amount + 1` or `max_swap_amount = swap_amount - 1` to narrow down the interval correctly.

```
match status {
    SwapResultStatus::Done => {
        #[cfg(test)]
        println!("Done calculate swap result {}", _i);
        break;
    }
    SwapResultStatus::ExceededA => {
        if trade_direction == TradeDirection::AtoB {
            // need to increase swap amount
        }
    }
}
```

```
        min_swap_amount = swap_amount;
    } else {
        // need to decrease swap amount
        max_swap_amount = swap_amount;
    }
}
SwapResultStatus::ExceededB => {
    if trade_direction == TradeDirection::AtoB {
        // need to decrease swap amount
        max_swap_amount = swap_amount;
    } else {
        // need to increase swap amount
        min_swap_amount = swap_amount;
    }
}
}
```

Recommendations:

This can be resolved as follows:

```
match status {
    SwapResultStatus::Done => {
        #[cfg(test)]
        println!("Done calculate swap result {}", _i);
        break;
    }
    SwapResultStatus::ExceededA => {
        if trade_direction == TradeDirection::AtoB {
            // need to increase swap amount
            min_swap_amount = swap_amount;
            min_swap_amount = swap_amount + 1;
        } else {
            // need to decrease swap amount
            max_swap_amount = swap_amount;
            max_swap_amount = swap_amount - 1;
        }
    }
    SwapResultStatus::ExceededB => {
        if trade_direction == TradeDirection::AtoB {
            // need to decrease swap amount
            max_swap_amount = swap_amount;
            max_swap_amount = swap_amount - 1;
        } else {
```

```
        // need to increase swap amount
        min_swap_amount = swap_amount;
        min_swap_amount = swap_amount + 1;
    }
}
```

Meteora: Resolved with [PR-36](#).

Zenith: Resolved by terminating binary search when $\text{amount_in} = \text{swap_in_amount}$, which means it is not converging.

[L-2] Zap won't allow for referral

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: High

Target

- /programs/zap/src/ultils/damm_v2_ultils.rs#L383

Description:

The Meteora DAMM allows for callers to pass a referral address. Part of the fee is then distributed to the referral.

```
let trading_fee: u64 = fee_amount.safe_sub(protocol_fee)?;

let referral_fee = if has_referral {
    safe_mul_div_cast_u64(
        protocol_fee,
        self.referral_fee_percent.into(),
        100,
        Rounding::Down,
    )?
} else {
    0
};
```

However, in the current implementation, the referral always gets set to None.

```
damm_v2::cpi::swap2(
    CpiContext::new(
        self.damm_program.to_account_info(),
        damm_v2::cpi::accounts::SwapCtx {
            pool_authority: self.pool_authority.to_account_info(),
            input_token_account,
            output_token_account,
            pool: self.pool.to_account_info(),
            token_a_vault: self.token_a_vault.to_account_info(),
            token_b_vault: self.token_b_vault.to_account_info(),
            token_a_mint: self.token_a_mint.to_account_info(),
```

```
token_b_mint: self.token_b_mint.to_account_info(),
token_a_program: self.token_a_program.to_account_info(),
token_b_program: self.token_b_program.to_account_info(),
event_authority: self.damm_event_authority.to_account_info(),
program: self.damm_program.to_account_info(),
payer: self.owner.to_account_info(),
referral_token_account: None, // TODO check whether it should be
some(damm_program)
    },
),
```

As a result, the Zap program cannot be used with referrals, which will disincentivize integrators from using it.

Recommendations:

We recommend allowing the referral address to be passed when calling the zap function for DAMM.

Meteora: Acknowledged.

[L-3] `calculate_swap_result()` should throw an error on zero output amount

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [damm_v2_utils.rs#L158-L173](#)

Description:

It is possible for `calculate_swap_result()` to return a zero output amount if the input amount is significantly small.

However, `calculate_swap_result()` does not handle this case and would still turn it as a successful swap. If the binary search converges with this swap, it will cause `zap_in_damm_v2` to perform a meaningless swap. Though the impact is minimal since the input amount is small.

Recommendations:

Consider throwing an error in `calculate_swap_result` when swap output amount is zero.

Meteora: Resolved with [PR-35](#).

Zenith: Verified. Resolved by skipping swap when swap input and swap output are zero.

[L-4] `calculate_swap_amount()` uses the `swap_amount_in` that fails `validate_swap_result`

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [damm_v2_utils.rs#L439-L444](#)

Description:

When `validate_swap_result()` fails, it breaks and then use the `swap_amount` for that swap result.

That is incorrect since it uses the `swap_amount_in` that failed validation.

It should instead use the previous `swap_amount_in` that had a successful `validate_swap_result()`.

```
} else {  
    #[cfg(test)]  
    println!("can't validate swap result {}", _i);  
  
    break; // if we can't validate swap result, then just break
```

Recommendations:

This can be resolved by using the previous `swap_amount_in` that had a successful `validate_swap_result()` when `validate_swap_result()` fails.

Meteora: Resolved with [PR-25](#).

Zenith: Verified. Resolved by only executing first liquidity add and skipping swap when swap computation fails.

[L-5] Potential overflow for amount_in in calculate_swap_amount()

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [damm_v2_utils.rs#L391](#)

Description:

In `calculate_swap_amount()`, the `amount_in` is an average of the `max_swap_amount` and `min_swap_amount`.

However, the addition was computed in u64 and this could overflow and revert when when `max_swap_amount` and `min_swap_amount` exceeds u64, despite having an average value within u64.

```
let amount_in = max_swap_amount.safe_add(min_swap_amount)?.safe_div(2)?;
```

Recommendations:

This can be avoided by scaling up to u128 for the addition.

Meteora: Resolved with [PR-27](#).

Zenith: Verified. Resolved by calculating the average using $\text{min} + (\text{max} - \text{min}) / 2$.

[L-6] Usage of zap/ledger with multiple pools simultaneously is impossible

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: High

Target

- /programs/zap/src/instructions/ledger_instructions.rs#L8-L14

Description:

In the current implementation, only one ledger can be created per user. This is a result of the seed generation in

```
#[account(  
  init,  
  seeds = [USER_LEDGER_PREFIX.as_ref(), owner.key().as_ref()],  
  payer = payer,  
  space = 8 + UserLedger::INIT_SPACE,  
  bump  
)]  
pub ledger: AccountLoader<'info, UserLedger>,
```

This will cause issues if a user tries to use the Zap program with multiple pools at once, since the ledger tracks only two token balances. This would lead to inconsistencies if the same one were used for different pools.

Recommendations:

We recommend including either a pool address or a nonce in the seed. As a result, a user can create multiple ledgers, each corresponding to a single pool.

Meteora: Acknowledged.

[L-7] Overflow possible in update_ledger_balances

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- /programs/zap/src/state/user_ledger.rs#L17-L33

Description:

The `update_ledger_balances` function is used to update the ledger between the steps of the zap. It is provided with the user's post- and pre-action balances. The intention is to add/subtract the delta of both from the ledger's amount. To ensure that no overflow occurs (as unsigned integers are used), the modification is done as follows:

```
self.amount_a = self
    .amount_a
    .safe_add(post_amount_a)?
    .safe_sub(pre_amount_a)?;
self.amount_b = self
    .amount_b
    .safe_add(post_amount_b)?
    .safe_sub(pre_amount_b)?;
```

First, the post amount is added, and afterwards the pre amount is subtracted. This leads to an issue when the user holds a balance close to `u64::max`, the maximum possible value for the token library. An exemplary scenario can showcase this:

1. User has a balance of `u64::max - 1000` and sets his ledges `amount_a = 5000`, so he wants to deposit 5000 tokens
2. Some action happens, and the user's balance changes to `u64::max - 3000` (for example, he deposits some funds)
3. Now the calculation will do the following $((5000 + u64::max - 1000) - u64::max - 3000)$

If we don't consider integer maximums, this would result in `amount_a = 3000` without any issues. However, we run into an issue in this step $(5000 + u64::max - 1000)$ in the actual

code. This will produce the intermediary result (`u64::max + 4000`) which will overflow the `u64` and lead to `safe_add()` panicking.

Recommendations:

We recommend casting the values to `u128`, doing the calculation, and then downcasting to `u64` again. This way, no overflow can occur, since the total supply is limited to `u64::max`. Even if the user holds all tokens, no error can occur.

Meteora: Resolved with [PR-37](#).

Zenith: Verified.

4.3 Informational

A total of 5 informational findings were identified.

[I-1] Typo in `utils` folder

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [/programs/zap/src/utils](#)

Description:

The `/programs/zap/src/utils` folder includes a typo in its name.

Recommendations:

We recommend renaming the folder to `utils`.

Meteora: Resolved with [PR-37](#).

Zenith: Verified.

[I-2] Typo in damm_v2_utils

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- /programs/zap/src/utils/damm_v2_utils.rs

Description:

The /programs/zap/src/utils/damm_v2_utils.rs file includes a typo in its name.

Recommendations:

We recommend renaming the file to damm_v2_utils.rs.

Meteora: Resolved with [PR-37](#).

Zenith: Verified.

[I-3] Bps Max is defined twice

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [/programs/zap/src/math/price_math.rs#L7](#)
- [/programs/zap/src/constants.rs#L33](#)

Description:

The price_math.rs file defines a BASIS_POINT_MAX variable.

```
const BASIS_POINT_MAX: i32 = 10000;
```

However, the same value is already defined in the constants.rs

```
#[constant]  
pub const MAX_BASIS_POINT: u16 = 10_000;
```

Recommendations:

We recommend using the definition in constants.

Meteora: Resolved with [PR-37](#).

Zenith: Verified.

[I-4] `calculate_swap_result()` hardcodes `has_partner = false` but does not verify it

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [damm_v2_utils.rs#L147](#)
- [damm_v2_utils.rs#L170](#)

Description:

`calculate_swap_result()` hardcodes `has_partner = false` when calling `pool.pool_fees.get_fee_on_amount()`.

However, this might not be true if the pool has configured partner fees.

```
let FeeOnAmountResult { amount, .. } = pool.pool_fees.get_fee_on_amount(
    excluded_fee_amount_in,
    trade_fee_numerator,
    fee_mode.has_referral,
    false,
)?;
```

Recommendations:

Consider validating that the pool has no partner fees configured.

Meteora: Acknowledged. We plan to remove partner fee in damm v2. Further more, partner fee is not affected on total trading fee (that user has to pay).

[I-5] pow() fails to handle the case `exp = i32::MIN`

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [price_math.rs#L42](#)

Description:

The `pow()` will perform a `exp.abs()` as `u32` when `exp` is negative.

However, it could result in an incorrect result when `exp = i32::MIN`, as the absolute value of `i32::MIN` cannot be represented as an `i32` and it could either trigger a panic or return `i32::MIN` as described [here](#).

Though this issue has no impact now as there are no instances where `exp` is negative when calling `pow()`.

```
// Make the exponential positive. Which will compute the result later by 1 /  
base^exp  
let exp: u32 = if invert { exp.abs() as u32 } else { exp as u32 };
```

Recommendations:

Consider returning `None` when `exp = i32::MIN`. Alternatively use `unsigned_abs()` instead.

Meteora: Resolved with [PR-34](#).

Zenith: Verified.